

Original Software Publication

A new drone chasing drone approach based on deep reinforcement learning with accelerated rewards

Ziya Tan^{a,*}, Mehmet Karaköse^b

^a Tokat Gaziosmanpaşa University Tokat, Turkey

^b Firat University Elazığ, Turkey



ARTICLE INFO

Keywords:

Deep reinforcement learning
Drone following
Drone detection
Deep deterministic policy gradient

ABSTRACT

In this paper, we propose a deep reinforcement learning-based approach that uses the drone camera as the only input source for a drone to track another drone in real-time autonomously. Deep learning and deep reinforcement learning algorithms are developed for this proposal. First, one of the object detection algorithms, YOLO, was trained with a dataset of different drone images to instantaneously detect drones in the images taken from the camera of the following drone. The detected drone was enclosed in a box and positioned on the screen. The size information of the box was sent to the agent trained with the Deep deterministic policy gradient (DDPG) deep reinforcement learning algorithm to determine the action of the following drone. This approach is tested in five different following scenarios. These two scenarios were also tested in environments with different light levels. According to the results of the scenarios, the following accuracy rate is calculated to be at most 99 %, and the drone response accuracy rate is calculated to be at most 95 %. In addition, the OpenAI Gym simulator is designed according to our problem, the DDPG agent is trained, the performance of the developed system is tested in a real-world environment, and the results are observed.

Metadata

C1	Current code version	V 1.0
C2	Permanent link to code/repository used for this code version	https://github.com/ElsevierSoftwareX/SOFTX-D-24-00498
C3	Permanent link to reproducible capsule	https://github.com/ziya-a44/Drone_tracking_with_drone
C4	Legal code license	MIT Licence
C5	Code versioning system used	None
C6	Software code languages, tools and services used	Python
C7	Compilation requirements, operating environments and dependencies	Ubuntu 20.04
C8	If available, link to developer documentation/manual	
C9	Support email for questions	ziya.tan@gop.edu.tr

1. Motivation and significance

Artificial intelligence (AI) [1] enables machines to learn from experience, adapt to new inputs, and perform human-like tasks. Most

examples of AI you hear about today - from computers playing chess to self-driving cars - are based on deep learning and natural language processing. Deep learning and reinforcement learning are the AI algorithms that bring these systems to life. Reinforcement learning (RL) [2] is one of the critical areas of machine learning. Using this approach, a computer program can be developed that learns what action to take in each environment to maximize an accumulated reward function. In other words, by rewarding the correct actions to be taken, the program learns how to act to solve the problem. Although this approach was first used in discrete and finite environments, today, solutions can be found in continuous environments with the development of function estimators and the introduction of artificial neural networks. In recent years, RL approaches have achieved remarkable results in many fields. Studies and results, especially in robotics, have been promising. In addition, the success of Atari games at the human level [3], the victories achieved with AlphaGo Zero [4], the performance of the DOTA2 game [5], and developments in autonomous vehicles bring RL to the forefront. In parallel with these developments, interest in UAVs is increasing daily. UAVs are developing thanks to their various applications in surveillance, disaster management, security, natural phenomena, the movie industry,

* Corresponding author.

E-mail addresses: ziya.tan@gop.edu.tr (Z. Tan), mkarakose@firat.edu.tr (M. Karaköse).

<https://doi.org/10.1016/j.softx.2025.102201>

Received 17 September 2024; Received in revised form 5 May 2025; Accepted 12 May 2025

Available online 17 May 2025

2352-7110/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

and the military.

In this study, we evaluate the superiority of DDPG, a deep reinforcement learning algorithm, and YOLO, a deep learning algorithm, for drone following with drone in dynamic and complex environments through actual experiments. This research aims to increase the effectiveness of drone technology in many industrial and commercial areas, especially in security and surveillance applications. By filling the existing gaps in the literature and proposing a new hybrid model and algorithm tested in real-world scenarios, it aims to make significant contributions to the knowledge and applications in this field.

In the study [6], a multi-sensor drone detection and tracking system is designed, and thermal infrared cameras and a fisheye camera are investigated in addition to standard image sources. It was concluded that the modifications improved the existing system and reduced false detections. In the article [7], which examines the threats caused by various UAVs and reviews the studies on the detection and tracking of foreign drones, computer vision techniques using radio frequency, radar, and various sensors for drone detection are examined. In [8], which presents deep reinforcement learning approaches for swarm navigation, training is performed using 3D simulations with different obstacles. The reward function developed for agent training is presented with policy-based deep learning algorithms. In the paper [9], where autonomous drone races are analyzed, information about the development of model-based learning approaches is presented. In [10], a distributed solution based on the proximal policy optimization (PPO) algorithm is proposed for a UAV to track another UAV autonomously. It is concluded that the distributed use of the PPO algorithm provides a practical benefit in terms of training performance and time. In [11], the development of an AirEye UAV-based platform is proposed to enable a drone to visit unlocated targets in a minimum amount of time with a probability calculation. In [12], a deep reinforcement learning approach is proposed to formulate a trajectory following a navigation system for quadcopters to follow the required trajectory. Jin-Hyeok Park et al. [13] presented a state-of-the-art object tracking framework using deep reinforcement learning algorithms in experiments on two training datasets. Experimental results show that the proposed approach is successful in speed and accuracy. In [14], a novel tracking algorithm for drone swarms is proposed by combining a learning network with another deep neural network. According to the test results, it was observed that the developed tracking model improved the performance by approximately 20 % compared to traditional tracking models. In [15], an innovative deep reinforcement learning algorithm for vertical take-off and landing of drones in windy environments is presented. The results showed that the drone gave successful results regarding flight stability. In the study [16], where the PPO algorithm is presented to optimize the speed control strategy of a quadcopter, four learned neural networks using end-to-end control commands are used. Thus, the accuracy of speed tracking is greatly improved. Moreover, the learning phase combines two phases: online and offline. In the study [17], which analyses the policies used in deep reinforcement learning algorithms, the policies used are grouped, and a report on their efficiency is presented. In addition, the training results of two agents targeting an everyday object using four different policies are also discussed. The paper [18] proposes a model-based reinforcement learning method based on a conditional variational autoencoder to minimize prediction errors in model-based reinforcement learning. The generative model is used to predict environmental variables. The data obtained in the study [19], which includes the model developed for the detection of node behaviors where parameters such as the number of packets and connection times performed by the attacker in a system are at the forefront, were used to train three different models such as feed-forward neural network, Cascade

back-propagation neural network, and Convolutional neural network.

A large data set was created regarding the drone detection algorithm required for following a drone and another drone with appropriate performance. Then, drone detection was performed using the YOLOv3 [33] object detection algorithm. It aims to develop a system that follows a target drone at an average speed of 6 cm/sec and performs at least 95 % accuracy in drone detection. We also tested the developed model in five scenarios and discussed the results. In addition, Table 1 provides a detailed analysis of the articles published in recent years on drone following.

This paper will contribute to the solution of two main problems. The first is the drone detection capability, and the second is the following of another detected drone. YOLO, one of the popular object detection algorithms, is optimized to detect the presence of a drone in the environment using the camera integrated into the follower drone. A dataset of approximately five thousand different images was prepared for the YOLO object detection algorithm, and training was performed with an accuracy of over 95 %. After successful target detection, the DDPG [34] algorithm trains the agent to follow the target drone autonomously. The following function is implemented in two different ways. The DDPG agent determines the up-down and yaw movements of the drone, and the forward-backward movements are determined by the size of the box framing the detected drone. The OpenAI-GYM simulator trained the follower drone to learn the following function. The reward function used during the training enabled the DDRL agent to choose the correct action by rewarding or penalizing it according to the coordinates of the bounding box detected by the object detection algorithm and given as input.

The experiments conducted in this study were carried out in a real-world environment. In this process, some difficulties were encountered considering the hardware capacity of the drone used as a follower drone and the technical specifications of the central computer. These challenges are the signal exchange between the tracker drone and the central computer is not strong enough, the maximum speed of the tracker drone is limited, and since the experiments are conducted indoors, the positive or negative effects of weather conditions and daylight on the tracker drone cannot be adequately measured. To overcome these challenges, deep learning, and reinforcement learning algorithms were trained according to the problem, and good results were obtained. Also, in this study,

- A state-of-the-art method for autonomous drone following is proposed,
- Drone detection, distance-location detection, following algorithms, and learning approaches have been developed for drone following.
- With deep reinforcement learning approaches, autonomous drones following action simulation and real-world tests were performed to validate with experimental data.

2. Software description

This section explains the details of the software used in our study, the characteristics of the experimental environments, and the basic concepts. Firstly, the formal representation and details of the proposed method are explained, and then the methods used for drone detection and drone following are presented. In addition, the parameters used in these methods are given in tables.

2.1. Software architecture

This study consists of two main phases for the real-time drone

Table 1
Comparative article analysis.

Article	Suggestion	Method	Result
[20]	A real-time optimization strategy is proposed to detect and track challenging targets in forested areas.	The study used Forest simulation where the ProcTree tree algorithm was implemented with WebGL.	The proposed approach achieved a target visibility of 72 % within a short time (14 s).
[21]	Wi-Drone is proposed to successfully realize drone flight control in environments where signal transmission is impossible.	Wi-Drone is a Wi-Fi standalone 6-DoF following system for indoor use. The experiments were performed in a real-world environment.	As a result of the experiments, the mAP of the Wi-Drones was measured as 3.8. It outperforms the VINS-Mono-LL algorithm by 42.7 % and the VINS-Mono-TL algorithm by 58.9 %.
[22]	This paper presents a critical overview of the drone detection approaches and state-of-the-art technology available in the literature.	A unique hybrid drone detection system has been developed in line with the investigations. The experiments were performed in a simulation environment.	The results show that the sound-assisted system (95.74 %) significantly improved the performance of the visual perception system (79.54 %).
[23]	An autonomous approach employing deep reinforcement learning is presented in this study for swarm navigation.	This study simulates 3D environments with different dynamic obstacles to follow multiple targets. A new island policy optimization model is developed for the following function. In this context, a new reward function is developed during the training phase of the deep reinforcement learning agent to improve target following and swarm behavior.	Experimental results demonstrate the success of the proposed approach in different situations that can be encountered in swarm actions. Compared to the existing techniques, the proposed framework improves the cumulative reward to a range of 1500–1600.
[24]	Drone navigation without GPS and in dynamic environments	A Kalman Filter with Unknown Input (KF-UI) was used to estimate the drone's linear velocities and the moving environment accelerations. The experiments were performed in a simulation and real-world environment.	According to the experimental results, the proposed approach successfully achieved the desired values despite some drawbacks.
[25]	A new real-time drone detection algorithm is proposed based on the YoloV3 object detection algorithm.	The CNN network structure in the existing YOLOv3 algorithm has been updated to improve the performance in detecting small-scale drones. In addition, the three-scale detection structure has been changed from three-scale to four-scale to improve the training performance.	According to the results, the new YOLOv3 achieved an average of 96 % and 95.6 % accuracy.
[26]	The YOLOv5 object detection algorithm is developed for drone detection. A new drone following method is also proposed.	The Kalman algorithm is trained to improve the UAV following performance. YOLOv5 was also improved to increase the detection probability. The experiments were performed in a simulation environment.	The experiments achieved about 2.8 % improvement in the UAV detection phase. In addition, the algorithm achieved a 4.4 % performance improvement in detecting small objects.
[27]	The autonomous navigation of a UAV is proposed. This framework provides agent training with an actor-critic-based model in a simulated environment.	The simulation determines the expected action by calculating the distance between the UAV and the target. The agent performing this action is trained using the Deep Deterministic Policy Gradient approach.	According to the proposed method, 79 % of the episodes were successful. Also, the average reward is 95.8 in general.
[28]	A new deep reinforcement learning-based control method for dynamic target following is proposed.	Two different methods were used. First, a DRL-based soft actor-critic (SAC) algorithm is developed using the onboard camera image. Second, a reward function is defined to improve the performance of this algorithm. The experiments were performed in a simulation environment.	Tracking reactions in the y-axis direction, where the target motion varies greatly, is significant. The absolute value of the maximum position tracking error in the y-axis direction is approximately 1.14 m. The average distance tracking error between the UAV and the target is about 0.61 m.
[29]	A lightweight Deep Vision Reinforcement Learning (LDVRL) framework is proposed for UAVs using drone cameras for object following in a dynamic environment.	DDPG and DQN deep reinforcement learning networks validated the proposed approach. The experiments were performed in a simulation environment.	The results of the tests show that DQN performs better. This is because the DQN architecture has fewer networks than the DDPG architecture.
[30]	Two different approaches are proposed for UAV tracking.	A deep reinforcement learning algorithm is developed to track the target drone. Furthermore, an object detection algorithm and a search space are proposed to predict the following action. The proposed algorithm is tested in real-world and simulator environments.	In particular, the search area proposal-based approach successfully detects small drones. After 150 epochs of training on 90 % of the images, an mAP score of 0.96 was obtained.
[31]	An approach to detect and track small UAVs in non-GPS environments is proposed.	The approach is tested in a real-world environment, and a Machine Learning model is developed for tracking drones at a given distance.	As a result of the tests, it was determined that the accuracy rate of the drone detection algorithm was 77 %.
[32]	The proposed method uses a deep reinforcement learning algorithm to successfully land a drone on moving ground.	The Deep Deterministic Policy Gradients (DDPG) algorithm is used to autonomously learn the landing maneuvers of the drone by modeling a partially observable Markov decision process (POMDP). The experiments are validated in the simulation engine.	According to the simulation results, it is concluded that the proposed algorithm gives 10 % more successful results compared to the PID method.
This Study	A hybrid approach of deep learning and reinforcement learning for drone tracking with drone	Training phases were carried out in a simulation environment, and test phases were carried out in a real-world environment.	In the experiments, the tracking accuracy rate was calculated at most 99 %, and the drone response accuracy rate was calculated at most 95 %.

following task. In the first phase, drone detection is performed. YoloV3 object detection algorithm is used for drone detection. In the second phase, the DDPG agent is trained to follow the detected drone. As a result of the training, the follower drone continues the following process by performing its action according to the target drone.

Fig. 1 shows the stages of the proposed approach. The figure shows the tasks the following drone performs, the drone being followed, and the central computer. The follower drone is wirelessly connected to the central computer. The first task of the follower drone is to detect an existing drone in the environment. When a drone enters the field of view of the follower drone, it is detected by the object detection algorithm and enclosed in a box. After determining the distance between the center of the detected drone's bounding box and the screen's center, the pre-trained DDPG reinforcement learning agent selects an action, and the tracking task is performed. This process continues as long as the tracked drone remains in the field of view of the follower drone. Without any drone in the field of view, the follower drone scans the environment again. If we write the process shown in Fig. 1 in more detail.

The system consists of three main components: the follower drone, the followed drone, and the central computer. The follower drone is responsible for detecting and following other drone in the environment, while the central computer processes image data and provides control commands.

1. Central Computer and Drone Detection Process

The follower drone is wirelessly connected to the central computer. The tracking process consists of the following steps:

Drone Detection:

- The follower drone utilizes the YOLO object detection algorithm (Drone Detection with YOLO) to identify the presence of other drones in its field of view.
- When a drone enters the field of view, it is detected by the YOLO algorithm and enclosed in a bounding box.
- The distance between the center of the detected drone's bounding box and the center of the screen is calculated.

- This information helps the follower drone determine its position relative to the target.

Processing Positional Data:

- The central computer computes the pixel coordinates of the detected drone to determine its position.
- If the drone is not centered in the field of view, the system generates motion control commands to adjust the follower drone's trajectory accordingly.
- This ensures that the follower drone stays aligned with the tracked target.

2. Tracking Process and Reinforcement Learning (RL) Implementation

The tracking mechanism is based on Deep Reinforcement Learning (DRL), specifically using a Deep Deterministic Policy Gradient (DDPG) agent. The tracking process is as follows:

The DDPG agent receives observations from the environment:

- These observations include information such as the position, orientation, and speed of the followed drone.
- The agent processes this data to determine how the follower drone should move.

The DDPG agent selects an appropriate action:

- If the followed drone moves left, right, up, or down, the follower drone executes corresponding movements to maintain alignment.
- These actions are determined based on the computed distance and directional offset of the detected drone.

Continuing the Tracking Process:

- If the followed drone remains within the field of view, the tracking process continues without interruption.

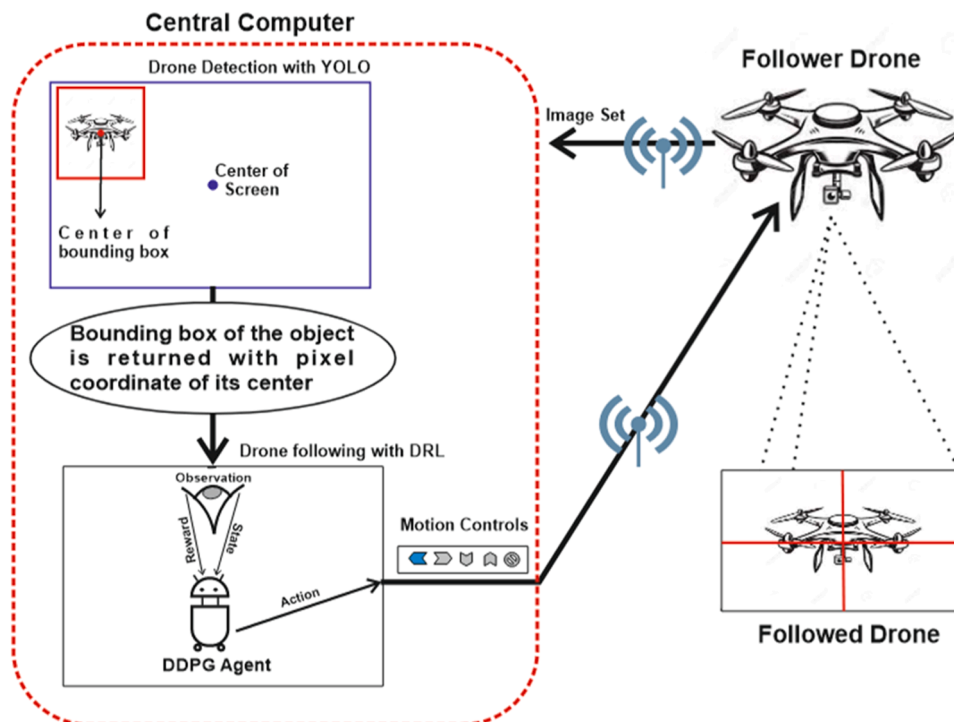


Fig. 1. Annotated framework of the proposed approach.

- If the followed drone moves out of sight, the follower drone initiates a rescan operation to reacquire the target.

3. Rescanning and Environment Scanning Mechanism

If the followed drone exits the field of view, the follower drone executes the following steps:

The follower drone begins scanning its surroundings.

- It first maintains its position and then adjusts its field of view to locate a new target.
- The scanning operation is performed according to a predefined movement pattern.

If a new drone is detected, the tracking process restarts.

- Once a new drone is identified, the system re-engages the object detection and tracking pipeline.
- If no drone is detected for an extended period, the follower drone switches to a predefined patrol mode to search for potential targets.

2.1.1. Drone detection

In this study, YOLOv3, one of the real-time object detection algorithms in the literature, is used for drone detection. The detected bounding box's size and position on the current screen will be influential in determining the position and direction of the following drone. The camera on the follower drone is utilized to detect the drone to be followed.

2.1.2. Drone following

In this project, the image frame taken from the drone is set to 960×720 pixels. Therefore, the center of the frame ($x = 480, y = 360$) will be fixed. The target drone's location in the image will be determined by calculating the center of the bounding box found by the deep learning model. As shown in Fig. 2, the follower drone will determine its actions in the horizontal and vertical planes according to the position of the detected drone. The follower drone determines its distance from the target drone according to the ratio of the detected bounding box to the

primary screen size. If this ratio is below 20 %, it is aimed to move forward; if it is above 55 %, it is aimed to move backward. If this ratio is between 20 % and 55 %, it will not move. Other actions (up, down, right, left) are determined by the agent trained by the DDPG algorithm. The working principle of the model is as follows;

The image captured by the follower drone is sent to the central computer. The captured image is transmitted to the object detection model running on the central computer. If there is a drone in the captured image, the bounding box is rotated along with its coordinates. The RL agent, trained by the DDPG algorithm on the central computer, determines the action of the follower drone by sending commands. As a result, the difference between the red and blue dots is minimized. With each snapshot, this process cycle starts all over again, as shown in Fig. 2

2.2. Software functionalities

This section describes the functions of the software used in the study for the following drone.

2.2.1. Deep reinforcement learning for drone following

This section describes the algorithms and basic concepts used in our study. Reinforcement Learning (RL), pioneered by Richard S. Sutton, is a machine learning approach that learns how to solve a task by predicting actions to be taken in an uncertain environment through trial and error [35]. Unlike other machine learning methods, no prior knowledge is given in RL theory. RL's goal is to maximize the rewards of future actions by using the experience gained after exploring the environment. There are two essential elements in RL: the agent and the environment. The two distinguishing features of RL are trial-and-error search and delayed reward [36]. These algorithms update the weights using stochastic gradient descent. The function that drives the action value functions is given in (1).

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \quad (1)$$

Usually, the value function estimate can be represented in a tabular form. However, the large and continuous state spaces that arise in real-world problems slow down training considerably. In this case, the problem is solved by using neural networks. The agent's goal is to

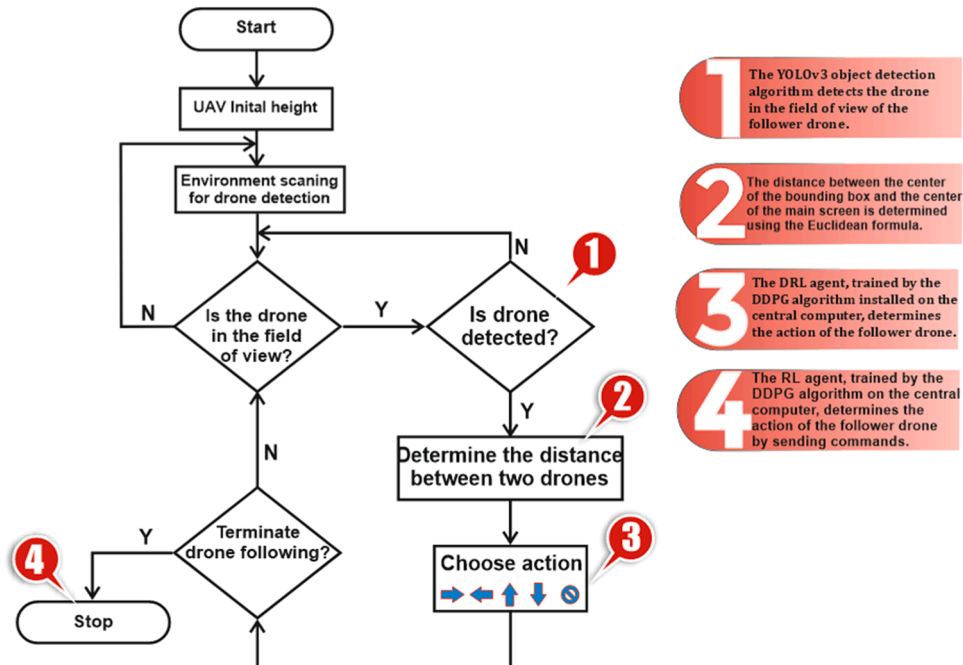


Fig. 2. Flow chart of the proposed approach.

maximize the cumulative reward, as shown in (2), and to converge the neural network to the optimal action-value function.

$$Q'(s, a) = \max_{\pi} \mathbb{E} \{ r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = a, \pi \} \quad (2)$$

$Q'(s, a)$ is the maximum possible return earned under any policy after moment t .

2.2.2. Deep deterministic policy gradient (DDPG)

Deep Deterministic Policy Gradients is an algorithm for learning continuous control policies in reinforcement learning environments. It is an actor-critic method, a class of RL algorithms that uses two separate neural networks, one for the "actor" (policy) and one for the "critic" (value function), to learn how to act in an environment to maximize a reward signal. The main goal of the critic network is to converge the estimated state-action value $Q(s, a)$ to the target value as closely as possible. We can define the target value as in (3).

$$y_i = r_i + \gamma Q'(S_{i+1}, \mu'(S_{i+1} | \theta^{\mu'})) \quad (3)$$

In (3), r_i is the active reward, S_{i+1} is the next state, Q' is the state-action value estimated by the target Critical Network, and μ' is the action chosen by the target actor network.

In (4), the equation that minimizes the loss function is shown, which is the objective of the Critical Network.

$$L = \frac{1}{N} \sum_i (y_i - Q(S_i, A_i | \theta^Q))^2 \quad (4)$$

One of the critical features of DDPG is that it uses a form of off-policy learning, which means that the algorithm can learn from experiences collected using a different policy than the one learned from. This allows DDPG to learn from a replay buffer that stores many past experiences and allows the algorithm to learn from them efficiently and effectively.

2.2.3. Reward function

The reward function we use in our approach could have been more specific, but it is formulated in a way that meets the basic requirements. First, we determine the distance between two points (the center points of the follower drone and the followed drone on the screen), which is the main parameter of the reward function. For the distance, the classical Euclidean equation is used. The Euclidean boundary calculates the distance between the box's center and the screen's center point. The drone detected in the image taken from the camera of the following drone is enclosed in a box by the object detection algorithm. According to the distance between this box's center point and the main screen's center point, the agent will receive a reward. The base distance value was determined as 110 units due to the experiments. The distance formula and reward function are used in (5) and (6).

$$dist = \sqrt{(state - goal_position)} \quad (5)$$

State is the drone's current position, and goal position is the screen's center point.

$$reward = \begin{cases} -dist * 0.25 & \text{if } dist > 110 \\ 100 - dist & \text{if } dist < 110 \end{cases} \quad (6)$$

The function used is a classic distance-based reward function. The goal is to incentivize the drone to get as close as possible to the center of the target.

- If the drone is exactly in the center of the target, the reward reaches its maximum value (e.g. 0 or close to it).
- If the drone is far from the target, the reward takes a very low value (e.g., -50).

This formula works with the Euclidean distance calculation:
Where:

- State → Location of the followed drone
- Goal_position → Position of the following drone

The smaller this distance, the closer the drone is to the target.

Strengths of the Function

Simple and effective: the reward increases as the distance decreases, i.e. the system learns to get closer to the target.

Low computational cost: Euclidean distance calculation can be run fast on CPU.

Focuses on direct target tracking: Guides the drone on an optimal trajectory, avoiding unnecessary movements.

Alternative reward function strategies have been considered, such as incorporating speed constraints or penalizing abrupt direction changes. However, these approaches have not been implemented due to the increased training complexity they introduce.

2.2.4. Hyperparameters of the software

Table 2 shows the parameter values used in the training phase. The training of the DDPG agent was performed on a computer with an Intel i5 processor and 8 GB RAM, utilizing only a CPU. These parameters were carefully selected through empirical testing and Bayesian optimization, ensuring optimal convergence and stable learning. The values used in training are listed in Table 2.

The selection of these hyperparameters followed a two-stage process:

Empirical Grid Search: Initial values were chosen based on standard reinforcement learning practices. Multiple training sessions were conducted with varying values to observe their impact on training stability and performance.

Bayesian Optimization: The key parameters (α for learning rate, γ for discount factor, and σ for exploration noise) were fine-tuned using Bayesian Optimization to maximize the cumulative reward. This optimization process ensured that the DDPG agent learned efficiently while avoiding overfitting or unstable convergence. A comparative analysis of different hyperparameter settings was conducted to evaluate their impact on model performance. The results are summarized in Table 3.

The selected hyperparameter set resulted in faster convergence and higher total reward, demonstrating a well-balanced trade-off between learning speed and stability. A lower discount factor ($\gamma=0.9$) led to short-term decision-making, reducing overall reward accumulation. A higher exploration noise ($\sigma=0.5$) caused excessive random actions, leading to unstable behavior.

3. Illustrative examples

To successfully complete this task, an approach is presented in which deep learning and reinforcement learning algorithms work

Table 2
Parameter values used in training.

Parameter	Value	Description
Learning rate for Actor and Critic	0.0001	Defines the step size for updating the neural network weights. Lower values prevent instability, while higher values speed up learning but may cause divergence.
Gamma(γ)	0.99	Discount factor determining the importance of future rewards. A value close to 1 prioritizes long-term rewards, while lower values focus on immediate gains.
Sigma(σ)	0.3	Exploration noise is added to actions to balance exploration and exploitation. Higher values encourage more random actions.
nb_max episode steps	10	Maximum number of steps per episode before termination. Ensures computational efficiency.
Number of episodes	100,000	Total training episodes. A higher number allows the agent to explore a larger state space and improve policy learning.
Interval	100	The frequency at which training progress is logged and evaluated.

Table 3
Impact of hyperparameters on training performance.

Parameter Set	Total Reward	Training Time (Episodes)	Stability
(Selected values: $\gamma=0.99$, $\alpha=0.0001$, $\sigma=0.3$)	-4.2	25,000	High
$\gamma=0.95$, $\alpha=0.001$, $\sigma=0.1$	-12.3	35,000	Medium
$\gamma=0.9$, $\alpha=0.0005$, $\sigma=0.5$	-20.1	50,000	Low

simultaneously. The results obtained are compared with similar studies in Table 4. Since few studies use similar methods, comparing the results has been limited. The hybrid system developed according to the following scenarios was successful for the test phase.

Table 4 compares the results of five drone following scenarios with results from similar papers. According to the table, the success of the approach we have presented can be seen.

As a result of the data analysis, the accuracy rates of the tracking algorithm and the variability on drone detection performance were analyzed. When the accuracy rates of the tracking algorithm are analyzed, it is seen that the mean of the data is 80.8 %, but the standard deviation is 0.2273 and the variance is 0.051. These values reveal that the tracking algorithm is highly accurate in some cases, but suffers from serious degradation in certain scenarios. In particular, the low accuracy value of 42 % increases the variability in the dataset and increases the standard deviation. If this low value is removed, the variance and standard deviation will decrease and the data set will become more stable. The performance of the drone detection algorithm has an average accuracy of 85.8 %. However, the standard deviation of 0.146 and variance of 0.021 indicate that the system is relatively stable, but in a particular scenario the accuracy drops to 60 %. These analyses show that the reasons for the system's low performance under certain conditions should be examined in detail and that it is important to optimize the algorithm accordingly.

Table 5 compares the flight time, tracking algorithm accuracy, and drone detection performance in different scenarios under low, medium, and high brightness conditions. When the data obtained for Scenario 2 and Scenario 3 are analyzed, it is seen that environmental factors significantly affect the system's performance.

1. Performance of the Tracking Algorithm:

- In Scenario 2, accuracy rates of 92 % at low brightness, 95 % at medium brightness, and 90 % at high brightness were achieved. This shows that the algorithm reaches the highest accuracy rate under medium brightness conditions, with slight decreases at low and high brightness levels.
- In Scenario 3, the accuracy ranges between 40 % and 55 %. At medium brightness, 42 % accuracy and 55 % at high brightness were achieved. The low accuracy rates indicate that the tracking algorithm suffers severe performance loss, especially in low-light conditions.

2. Drone Detection Performance:

- In Scenario 2, 92 % accuracy was achieved in low brightness, 94 % in medium brightness and 89 % in high brightness. The best results were obtained in medium brightness conditions, with slight differences between low and high brightness levels.
- In Scenario 3, accuracy rates of 55 % at low brightness, 60 % at medium brightness, and 52 % at high brightness were obtained.

Table 4
Comparison table of scenarios.

Scenario	Flight Duration (min)	Following the algorithm (accuracy rate)	Drone detection algorithm (Time of Flight/Accurate Detection)
1	4	% 99	% 95
2	9	% 95	% 94
3	10	% 86	% 90
4	9	% 82	% 90
5	5	% 42	% 60
[31]	1.8		%77
[32]		% 90.50	
[37]		%88.4	
[38]		%93.3	
[39]		%87.9	

Medium brightness conditions also provided the best results here, with significant decreases at low and high brightness levels.

General Evaluation:

- In Scenario 2, the tracking algorithm and the drone detection performance are high and operate in the 89 %–95 % accuracy range.
- In Scenario 3, on the other hand, low accuracy rates were observed and the accuracy of the tracking algorithm dropped to 40 %, especially in low brightness conditions.
- Medium brightness conditions generally provide the best accuracy rates, suggesting that the system performs best in balanced light conditions.
- Since significant performance loss was observed at low light levels, additional optimizations are required to enable the system to operate in low brightness conditions. This can be achieved by improving image processing algorithms, increasing the dataset, or training a model that works better in low light.

The results obtained within the scope of the study were analyzed using the accuracy rates of the tracking algorithm and the standard deviation and variance values for drone detection performance. In Scenario 2, the accuracy rate of the tracking algorithm was 92.33 % on average, with a standard deviation of 2.08 and a variance of 4.33. However, in Scenario 3, the accuracy rate dropped to 45.67 %, with a standard deviation of 7.64 and a variance of 58.33. This high variance and standard deviation indicate that the tracking algorithm varies significantly in low-light conditions.

Regarding drone detection performance, in Scenario 2, the average accuracy was 91.67 % with a standard deviation of 2.52 and a variance of 6.33, similar to the tracking algorithm. In Scenario 3, the accuracy was 55.67 %, with a standard deviation of 4.04 and a variance of 16.33. These results show that the drone detection algorithm is generally more stable but shows a certain sensitivity to light changes.

4. Impact

This study has conducted an experiment in which the test phases are carried out in a real environment for an autonomous drone followed by a drone. The results of the hybrid approach in which deep reinforcement learning and deep learning algorithms are used together in five different scenarios are presented separately and supported by graphs. In particular, the speed and instantaneous movements of the target drone were found to be effective in the success of the follower drone's mission. Another important factor was the adequacy of the hardware and

Table 5
Experiments in different brightness environments.

Env. Scnr.	Flight Duration (min)			Following the algorithm (accuracy rate)			Drone detection algorithm (Time of Flight/Accurate Detection)		
	Low-brightness	Medium-brightness	High-brightness	Low-brightness	Medium-brightness	High-brightness	Low-brightness	Medium-brightness	High-brightness
2	8	9	11	% 92	% 95	% 90	% 92	% 94	% 89
3	5	5	6	% 40	% 42	% 55	% 55	% 60	% 52

physical characteristics of the follower drone. The speed of communication with the central computer and the capacity of the central computer to process the received images were effective in the speed of the tracking drone to perform its actions. The current experiments were conducted only in an indoor gymnasium. In real-world applications, drones can be affected by external factors such as wind, rain, variable lighting conditions, sudden temperature changes and GPS signal interruptions. However, the study did not analyze the impact of such environmental factors on the system. Outdoor conditions can cause the following problems;

- Wind and Turbulence: It can make drone movements unstable, reducing tracking accuracy.
- Rain and Humidity: Can obstruct the camera's view and damage electronic components.
- Variable Light and Shade Problems: Sunlight at different times of the day can degrade the performance of the tracking algorithm.

The conducted experiments were performed in an indoor sports hall to ensure controlled testing conditions and minimize unpredictable environmental disturbances such as wind and rain. This setup allowed us to focus primarily on evaluating the effectiveness of our tracking algorithm under consistent conditions. However, real-world applications require robustness against environmental variations, including outdoor wind conditions, sudden lighting changes, and potential occlusions due to obstacles. One major limitation of this study is the lack of testing in outdoor environments. Real-world applications of drone tracking often involve external disturbances such as wind, rain, and fluctuating light conditions. These factors can significantly impact tracking performance by causing motion instability, sensor inaccuracies, and variations in object detection confidence.

Experiments were carried out indoors to get maximum efficiency from the follower drone. One of the main reasons for this is that the follower drone is lightweight and will be maximally affected by wind gusts. It would be difficult for a drone of this weight to perform the tracking function outdoors.

4.1. Experimental results

This section discusses the training results of the deep learning and deep reinforcement learning algorithms used in the paper. All results and details are presented in [Appendix A](#). First, the results of the drone detection algorithm are presented, and then the results of the following algorithm are presented. In addition, the proposed approach is tested in five different drones in the following scenarios, and the results are presented in detail.

5. Conclusions

According to the training results of the object detection algorithm, an accuracy rate of approximately 95 % was achieved. The DDPG agent is

trained in a simulation environment prepared in advance to follow the drone and determine the actions of the following drone. After 500 training steps were performed, it was concluded that the loss and reward values were at the desired level. The action is determined by the box that frames the detected drones. The follower drone detects drones instantly from the images taken from its camera and moves the main screen's center point closer to the detected drone's center point. The time required for action prediction has limitations depending on the hardware features of the follower drone and the system. These limitations are evident in the five scenarios that test the following drone. According to these scenarios, increases in the speed of the followed drone and sudden changes in its trajectory affect the following drone's tracking success. In these experiments, the follower drone was tested at the maximum level in terms of hardware capability. According to the results given in [Table 3](#), the follower drone was successful in all but one scenario.

The results in [Table 4](#) show that the tracking and detection drone algorithms are also successful in environments with different light levels. According to the experiments and results discussed above, we have performed well using the DDPG algorithm. We have also proved the applicability of our model by conducting tests in a real-world environment. Thanks to the network structure of DDPG and the correctly tuned hyperparameters, we successfully performed the following task on the drone. Due to the physical and hardware characteristics of the following drone, we completed the real-world tests in a closed area.

As a result of the gains obtained in this study, our future studies aim to develop a system that will enable autonomous following and coordinated movement for swarm drones. In the studies to be carried out in this direction, it will be ensured that it can instantly detect multiple drones around it and identify the drones that need to be followed within this drone swarm. At this stage, the aim is to develop a friend/enemy drone detection algorithm and perform the correct following autonomously. Ultimately, our goal is to design drone swarms that maintain autonomous following. In addition, existing national/international support will be utilized to provide financial resources.

To address wind-induced instability, onboard IMU (Inertial Measurement Unit) and barometer data will be integrated with the reinforcement learning model to compensate for unpredictable air movements. Additionally, adaptive exposure and contrast enhancement techniques will be applied to the object detection pipeline to improve accuracy in varying lighting conditions. By incorporating these strategies, we aim to enhance the adaptability of the tracking system for real-world drone applications.

CRedit authorship contribution statement

Ziya Tan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Funding acquisition, Formal analysis, Data curation. **Mehmet Karaköse:** Writing – review & editing, Validation, Software, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study was prepared by Ziya TAN's Ph.D. thesis titled "Distributed Deep Reinforcement Learning Approaches for Drone Tracking with Drone [40]".

This work was supported by the FIRAT University Scientific Research Projects Unit (FUBAP) under Grant MF.24.18.

Appendix A. Experimental results

Drone detection

The first step for autonomous drone following is to detect the presence of drones in the images. For this purpose, YoloV3 was trained using approximately 6500 different drone images. After 100 epochs of training, approximately 95 % accuracy was achieved. The training was completed using a batch size of 20 and a learning rate of 0.0001. Since the number of classes to be learned in the training phase was 1, high success was achieved with these parameters. It is seen that the accuracy rate given for the detected drone is lower in small images. To solve this problem, increasing the number of small drone images in the dataset used in the training phase would be sufficient.

The graphs of the values obtained from the training are shown in Fig. A.1. As shown in Fig. A.1, the loss value decreases rapidly during training while the precision value increases at the desired rate.

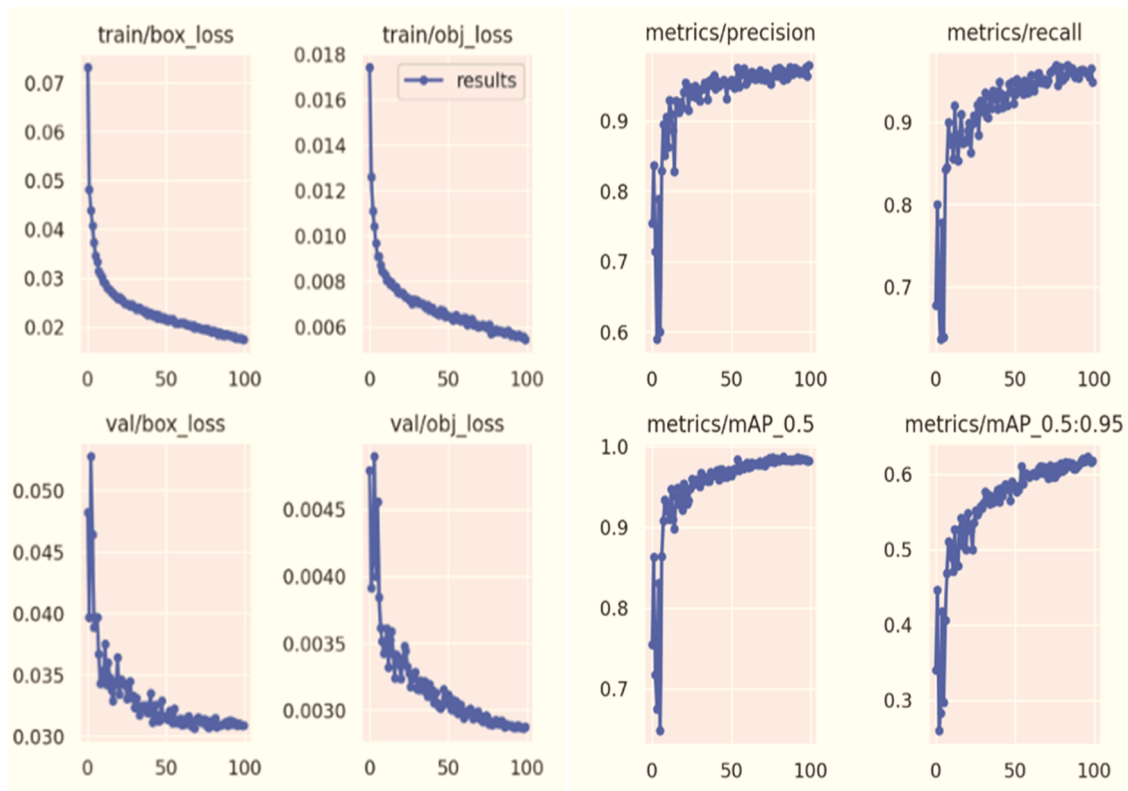


Fig. A.1. Object detection training results.

Drone following

The second step in autonomous drone following is to act as the following drone according to the position of the detected object on the main screen. For this purpose, training was carried out in the environment prepared using the GYM simulator environment.

The training lasted approximately four hours with the parameter values specified in Table 2. The graphs of the reward and loss values at the end of the training are shown in Fig. A.2(a) and Fig. A.2(b).



Fig. A.2. a) Reward graph because of training b) Loss graph because of training.

In this paper, experiments were conducted in a real-world environment for the test phases of the trained DDPG agent. In this phase, a DJI-Tello drone was used as a follower drone. The specifications of the drones used in this study are given in [Table A.1](#).

Table A.1

Hardware specifications of the follower drone.

Type of Drone	Company	Weight(g)	Endurance (min)	Max Speed (cm/sec)
Quadcopter	DJI-Tello(follower)	80	13	8
Quadcopter	Standard Drone(followed)	147	15	15

The following drone was a standard drone that can be controlled with a remote control. The test phases were carried out in an indoor sports hall, as shown in [Fig. A.3\(a\)](#). One of the reasons for performing the test environments in an indoor area is to prevent drones from being affected by wind and external factors. The results of the five following scenarios were calculated based on the flight times and the time the follower drone detected the followed drone and kept it in the field of view.

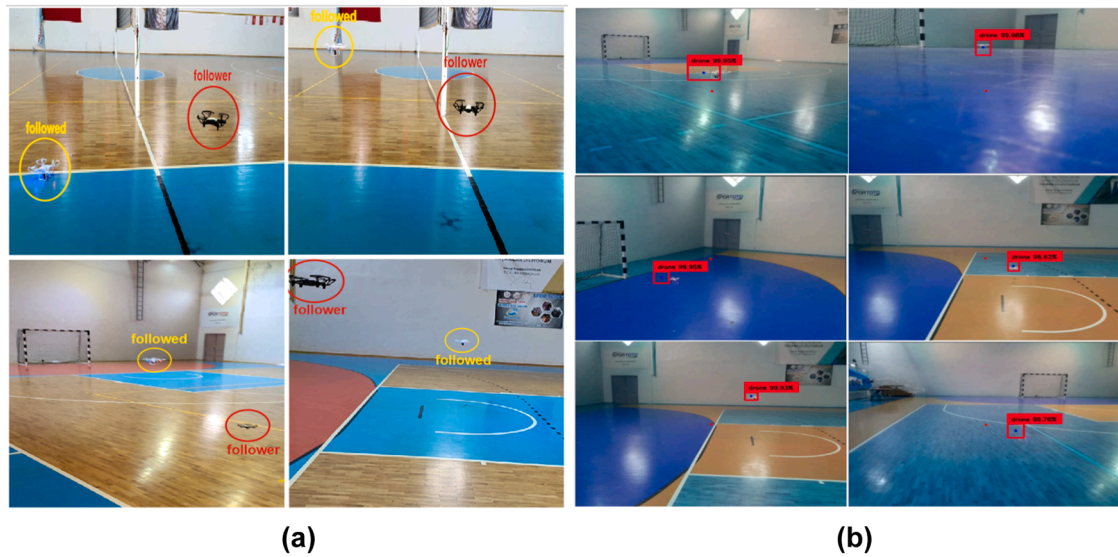


Fig. A.3. a) Examples from the real-world environment during following b) Snapshots from the camera integrated into the follower drone during the test phase.

In the test phase of the proposed method, snapshots from the camera integrated into the follower drone during the following are shown in Fig. A.3 (b). In some images, it is seen that the tracked drone is outside the bounding box. This is due to the capacity of the wireless communication between the Tello drone and the computer. Since the followed drone is moving, this difference is caused by the slight delay in the image transferred to the computer, although it is detected in some sudden accelerations.

One of the primary limitations encountered in this study was the latency caused by image processing and wireless communication between the tracking drone and the central computer. The delay in transferring images to the processing unit led to occasional inaccuracies in real-time tracking, especially in high-speed scenarios.

Future implementations will focus on integrating edge computing capabilities directly on the tracking drone to mitigate these issues. Using lightweight deep learning models optimized for NVIDIA Jetson modules or similar edge AI hardware will enable real-time inference without relying on external computation resources. Switching to 5G-based communication protocols or dedicated RF modules with lower latency will enhance responsiveness. These improvements will significantly reduce processing delays, allowing the system to function more efficiently in real-world applications.

Scenario 1: The first scenario aims to detect and follow a stationary drone at a distance of 25 m. After the drone takes off, the follower searches around the drone to detect its presence. After the drone was detected, it reached the target after about 4 min. During the following, the drone detection algorithm reached the target with an average rate of approximately 95 %. In addition, during the following period, the followed drone stayed within the field of view of the follower drone 99 % of the time. Fig. A.4(a) shows the following function visualization for the first scenario.

Scenario 2: In the second scenario, the following drone took a route at a constant speed, as shown in Fig. A.4(b). The two drones took off simultaneously and stayed in the air for about 9 min. During this time, the follower drone managed to keep the followed drone in its field of view approximately 95 % of the time from the first detection moment to the end of the pursuit. The drone detection algorithm achieved an average detection rate of approximately 94 % throughout the following.

Scenario 3: In the third scenario, the followed drone performed its flight by oscillating at variable speeds and irregular time intervals on the vertical axis. A representative flight path is shown in Fig. A.4(c). During the flight time of approximately 10 min, the follower drone kept the followed drone in its line of sight 86 % of the time. The followed drone left the field of view at the blue dots shown in Fig. A.4(c). During this period, the follower drone switches to search mode and scans the environment by rotating around its axis. The drone detected during the scan is followed again. The follower drone's average detection rate during the following was approximately 90 %.

Scenario 4: In this scenario, the followed drone flew at variable speeds and oscillated irregularly on the horizontal axis. A representative flight path is shown in Fig. A.4(d). During the flight time of approximately 9 min, the follower drone could keep the followed drone in its line of sight 82 % of the time and maintain the following distance. The blue dots in Fig. A.4(d) indicate the moments when the followed drone was out of sight. During this period, the follower drone circled to detect the presence of drones around it. The drone detected during the scan was followed again, and the task was completed. The average drone detection rate during the following was approximately 90 %. In this scenario, horizontal movements of the followed drone out of the field of view and changes in its speed were influential in determining the following performance.

Scenario 5: In this scenario, the followed drone moves at variable high speeds on the route shown in Fig. A.4(e). In the experiment, which lasted about 5 min, the follower drone was more unsuccessful than in the other scenarios. During this flight, it could keep the followed drone in its line of sight about 42 % of the time. The drone detection algorithm detected the followed drone at a rate of 60 %.

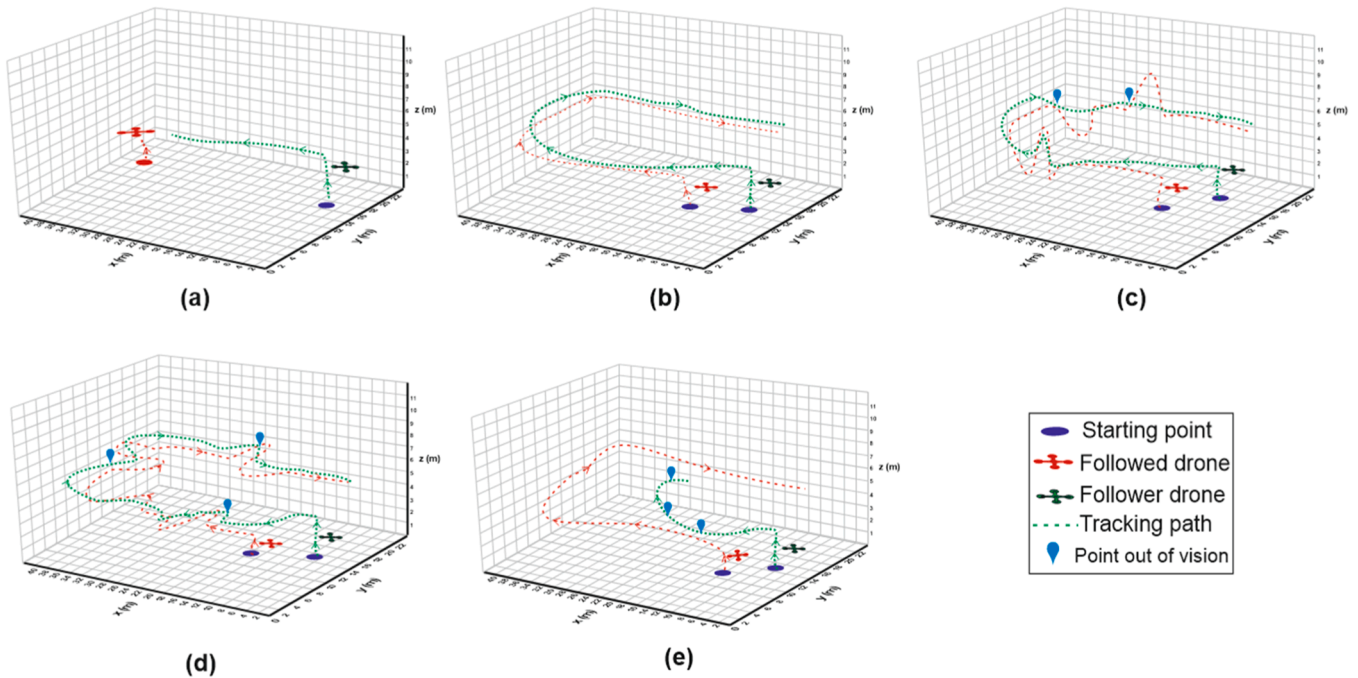


Fig. A.4. a) The following graph for Scenario 1 b) The following graph for Scenario 2 c) The following graph for Scenario 3 d) The following graph for Scenario 4 e) The following graph for Scenario 5.

Factors contributing to the follower drone's inability to follow its target accurately are. The followed drone makes irregular movements, and the detection algorithm makes false positives and negatives. It was observed that the drone deviated from its route due to the perception of different shapes in the environment in which the scenarios were performed, thus performing poorly in some periods. In the case of false- positives detected during the flight, the follower drone approached another object. Still, the followed drone remained in the field of view, allowing it to reorient to the correct target. This flight could keep the followed drone in its line of sight about 42 % of the time. The drone detection algorithm detected the followed drone at a rate of 60 %.

References

- [1] Winston PH. Artificial intelligence. 2nd ed. USA: Addison-Wesley Longman Publishing Co., Inc.; 1984.
- [2] François-Lavet V, et al. An introduction to deep reinforcement learning. *Found Trends Mach Learn* 2018;11(3–4):219–354.
- [3] Mnih V, et al. Human-level control through deep reinforcement learning. *Nature* 2015;518(7540):529–33.
- [4] Silver D, et al. Mastering the game of go without human knowledge. *Nature* 2017; 550(7676):354–9.
- [5] Xu Z, Tang C, Tomizuka M. Zero-shot deep reinforcement learning driving policy transfer for autonomous vehicles based on robust control. In: 2018 21st International Conference on Intelligent Transportation Systems (ITSC). IEEE; 2018. p. 2865–71.
- [6] Svanström F, Alonso-Fernandez F, Englund C. Drone detection and tracking in real-time by fusion of different sensing modalities. *Drones* 2022;6(11). <https://doi.org/10.3390/drones6110317>. Art. no. 11Nov.
- [7] Guvenc I, et al. Detection, tracking, and interdiction for amateur drones. *IEEE Commun Mag* 2018;56(4):75–81. <https://doi.org/10.1109/MCOM.2018.1700455>. Apr.
- [8] Qamar S, et al. Autonomous drone swarm navigation and multi-target tracking in 3d environments with dynamic obstacles. *IEEE Access* 2022;10:91073–91. <https://doi.org/10.1109/ACCESS.2022.3202208>.
- [9] D. Hanover et al., "Autonomous drone racing: a survey." arXiv, Jan. 05, 2023. doi: 10.48550/arXiv.2301.01755.
- [10] Tan Z, Karaköse M. A new approach for drone tracking with drone using Proximal Policy Optimization based distributed deep reinforcement learning. *SoftwareX* 2023;23:101497. <https://doi.org/10.1016/j.softx.2023.101497>. Jul.
- [11] Soliman A, et al. AirEye: uAV-based intelligent drl mobile target visitation. In: 2022 International Wireless Communications and Mobile Computing (IWCMC); 2022. p. 554–9. <https://doi.org/10.1109/IWCMC55113.2022.9824676>. May.
- [12] Bala K, Verma AS. Advanced Machine learning detection using UAV Motion Tracking and Control. In: 2023 International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE); 2023. p. 1–6. <https://doi.org/10.1109/ICDCECE57866.2023.10150563>. Apr.
- [13] Park J-H, Farkhodov K, Lee S-H, Kwon K-R. Deep reinforcement learning-based DQN agent algorithm for visual object tracking in a virtual environmental simulation. *Applied Sciences* 2022;12(7). <https://doi.org/10.3390/app12073220>. Art. no. 7Jan.
- [14] Fu W, Gu M, Niu Y, editors. Lecture notes in electrical engineering. Singapore: Springer Nature; 2023. p. 1047–61. https://doi.org/10.1007/978-981-99-0479-2_95.
- [15] Ma B, et al. Target tracking control of UAV through deep reinforcement learning. *IEEE Trans Intell Transp Syst* 2023;24(6):5983–6000. <https://doi.org/10.1109/TITS.2023.3249900>. Jun.
- [16] Hu H, Wang Q. Proximal policy optimization with an integral compensator for quadrotor control. *Front Inform Technol Electron Eng* 2020;21(5):777–95. <https://doi.org/10.1631/FITEE.1900641>. May.
- [17] Tan Z, Karaköse M. Comparative evaluation for effectiveness analysis of policy based deep reinforcement learning approaches. *International Journal of Computer and Information Technology* 2021;10(3). <https://doi.org/10.24203/ijcit.v10i3.104>.
- [18] Zhu T, Ren R, Li Y, Liu W. A model-based reinforcement learning method with conditional variational auto-encoder. *JDSIS* 2024. <https://doi.org/10.47852/bonviewJDSIS42022432>. Mar.
- [19] Yonan JF, Zahra NAA. Node intrusion tendency recognition using network level features based deep learning approach. *Babylonian J Network* 2023;1–10. <https://doi.org/10.58496/BJN/2023/001>. Jan. 2023.
- [20] Amala Arokia Nathan RJ, Kurmi I, Bimber O. Drone swarm strategy for the detection and tracking of occluded targets in complex environments. *Commun Eng* 2023;2(1). <https://doi.org/10.1038/s44172-023-00104-0>. Art. no. 1Aug.
- [21] Chi G, et al. Wi-drone: wi-fi-based 6-DoF tracking for indoor drone flight control. In: Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services, in MobiSys '22. Association for Computing Machinery; 2022. p. 56–68. <https://doi.org/10.1145/3498361.3538936>. Jun.
- [22] Khan MA, Menouar H, Eldeeb A, Abu-Dayya A, Salim FD. On the detection of unauthorized drones—Techniques and future perspectives: a review. *IEEE Sens J* 2022;22(12):11439–55. <https://doi.org/10.1109/JSEN.2022.3171293>. Jun.
- [23] Qamar S, et al. Autonomous drone swarm navigation and multitarget tracking with island policy-based optimization framework. *IEEE Access* 2022;10:91073–91. <https://doi.org/10.1109/ACCESS.2022.3202208>.
- [24] Marani Y, Telegenov K, Feron E, Kirati M-TL. Drone reference tracking in a non-inertial frame using sliding mode control based Kalman filter with unknown input. In: 2022 IEEE Conference on Control Technology and Applications (CCTA); 2022. p. 9–16. <https://doi.org/10.1109/CCTA49430.2022.9966119>. Aug.
- [25] Alsanad HR, Sadik AZ, Ucan ON, Ilyas M, Bayat O. YOLO-V3 based real-time drone detection algorithm. *Multimed Tools Appl* 2022;81(18):26185–98. <https://doi.org/10.1007/s11042-022-12939-4>. Jul.

- [26] Jiang Y, Jingliang G, Yanqing Z, Min W, Jianwei W. Detection and tracking method of small-sized UAV based on YOLOv5. In: 2022 19th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP); 2022. p. 1–5. <https://doi.org/10.1109/ICCWAMTIP56608.2022.10016550>. Dec.
- [27] Zhou B, Wang W, Liu Z, Wang J. Vision-based navigation of UAV with continuous action space using deep reinforcement learning. In: 2019 Chinese Control And Decision Conference (CCDC); 2019. p. 5030–5. <https://doi.org/10.1109/CCDC.2019.8832593>. Jun.
- [28] Zhao J, et al. Deep reinforcement learning-based end-to-end control for UAV dynamic target tracking. *Biomimetics* 2022;7(4). <https://doi.org/10.3390/biomimetics7040197>. Art. no. 4Dec.
- [29] Nguyen H, Thudumu S, Du H, Mouzakis K, Vasa R. UAV dynamic object tracking with lightweight deep vision reinforcement learning. *Algorithms* 2023;16(5). <https://doi.org/10.3390/a16050227>. Art. no. 5May.
- [30] Akhloufi MA, Arola S, Bonnet A. Drones chasing Drones: reinforcement learning and deep search area proposal. *Drones* Sep. 2019;3(3). <https://doi.org/10.3390/drones3030058>. Art. no. 3.
- [31] Wyder PM, et al. Autonomous drone hunter operating by deep learning and all-onboard computations in GPS-denied environments. *PLoS One* 2019;14(11): e0225092. <https://doi.org/10.1371/journal.pone.0225092>. Nov.
- [32] Xie J, Peng X, Wang H, Niu W, Zheng X. UAV autonomous tracking and landing based on deep reinforcement learning strategy. *Sensors* 2020;20(19). <https://doi.org/10.3390/s20195630>. Art. no. 19Jan.
- [33] Liu C, Tao Y, Liang J, Li K, Chen Y. Object detection based on YOLO network. In: 2018 IEEE 4th information technology and mechatronics engineering conference (ITOE). IEEE; 2018. p. 799–803.
- [34] Liu Y, Zhang W, Chen F, Li J. Path planning based on improved Deep Deterministic Policy Gradient algorithm. In: 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC); 2019. p. 295–9. <https://doi.org/10.1109/ITNEC.2019.8729369>. Mar.
- [35] Sutton RS, Barto AG. Reinforcement Learning: an introduction. *IEEE Trans Neural Networks* 1998;9(5). <https://doi.org/10.1109/TNN.1998.712192>. 1054–1054Sep.
- [36] V. Mnih et al., “Playing Atari with Deep Reinforcement Learning.” *arXiv*, Dec. 19, 2013. doi: 10.48550/arXiv.1312.5602.
- [37] Cheng Feng, et al. An anti-UAV long-term tracking method with hybrid attention mechanism and hierarchical discriminator. *Sensors* 2022;22(10):3701.
- [38] Zhao Jie, et al. Vision-based anti-uav detection and tracking. *IEEE Trans Intell Transp Syst* 2022;23(12):25323–34.
- [39] Wang Chuanyun, et al. Anti-occlusion UAV tracking algorithm with a low-altitude complex background by integrating attention mechanism. *Drones* 2022;6(6):149.
- [40] Tan Z. PhD Thesis. Elazığ: Institute of Science and Technology, Firat University; 2024.